PantryPal

MVP Report - MAE 301

Michael Franco, Suhayl Kudbudeen, Jose Ignacio Rozas
Hyeonji Sim, Erik Zamora

Table of Contents

Table of Contents 1

Executive Summary 2

User & Use Case 3

System Design 4

Data 5

Models 6

Evaluation 7

Limitations & Risks 8

Next Steps 9

Conclusion 10

Demo Evidence and Test cases 11

Executive Summary

PantryPal is an AI-powered web application designed to help users decide what meals they can prepare using ingredients they already have at home. Many people struggle with meal planning despite having available food, which often leads to wasted groceries or unnecessary spending.

The system allows users to enter a flexible list of ingredients and generates three meal suggestions. Each suggestion includes a short description, the ingredients used, any additional ingredients needed, and basic cooking details such as time and difficulty.

The MVP includes two implementations: a rule-based baseline and an AI-assisted version. These were tested using similar inputs to compare reliability and flexibility. The AI model helps interpret natural language input and generate more flexible meal ideas, while the rule-based system provides consistent baseline results. This MVP demonstrates that PantryPal can process unstructured input and produce useful meal suggestions.

User & Use Case

PantryPal is designed for users who want quick and practical meal ideas without needing to search through recipes manually. This includes college students, busy individuals, and anyone trying to reduce food waste or save money.

A typical use case involves a user opening their fridge and seeing a random set of ingredients, such as eggs, rice, and onions. Instead of searching for recipes online, the user enters these ingredients into PantryPal. The system then generates several meal suggestions that can be made with those ingredients, along with any additional items needed.

The system is designed to handle informal input, allowing users to type ingredients naturally rather than in a strict format. This makes the tool easy to use and helps users quickly decide what to cook.

System Design

PantryPal follows a simple but effective pipeline that takes user input, processes it, and generates meal suggestions. The user enters ingredients through a web interface, which sends the input to a backend server built with Python and Flask.

The backend processes the input in multiple steps. First, the input is cleaned and normalized to handle variations in how users describe ingredients. This includes splitting the input text, removing extra spaces, and standardizing ingredient names so they can be compared more effectively.

The system then uses two approaches to generate results. In the no-AI version, the backend compares the input to a local recipe dataset using ingredient matching and scoring logic to find the best matches. In the AI-assisted version, the backend sends the input to an AI model, which interprets the ingredients more flexibly and generates structured meal suggestions.

The outputs from both approaches are formatted and returned to the frontend, where they are displayed as interactive meal cards. Each suggestion includes a meal name, a short description, and any additional ingredients required. The system typically returns three suggestions to keep the results simple and useful.

This hybrid design allows PantryPal to combine the consistency of rule-based matching with the flexibility of AI-generated suggestions, while still handling unstructured user input effectively.

Data

PantryPal uses two main sources of data. First, a small curated dataset of approximately 50 recipes is used for the rule-based system. Each recipe includes ingredient lists, required ingredients, and additional metadata such as cooking time and difficulty. This dataset allows the backend to compare user input with existing recipes and supports fast, consistent matching achieving high accuracy.

Before using the dataset, basic cleaning and preprocessing were performed. This included standardizing ingredient names, removing duplicate or incomplete entries, and ensuring that ingredient lists were formatted consistently. These steps improve the accuracy of ingredient matching and overall system reliability.

Second, the AI-assisted system leverages a pre-trained language model that has been trained on large-scale text data. This allows the system to generate more flexible and context-aware meal suggestions without relying entirely on the local dataset.

User input is also preprocessed before even being used. This includes normalization techniques such as converting text to lowercase, removing unnecessary words, and basic combing through of ingredient lists. These steps help the system handle input and improve both matching accuracy and AI interpretation.

Also, custom test inputs were created to simulate real behavior. These inputs include ingredients such as “eggs, rice, random vegetables, etc ” to evaluate how well the system handles flexible input.

Models

PantryPal uses a hybrid modeling approach that combines a rule-based system with an AI-assisted generative model to produce meal suggestions.

The primary component of the system is a generative AI model accessed through Claude, which is an anthropic API. The user's ingredient list is passed as input, and prompt engineering is used to guide the model to produce structured outputs in a consistent JSON format. The prompt instructs the model to generate exactly three realistic meal suggestions while prioritizing the ingredients provided by the user. Each output includes a meal name, description, and any additional ingredients required. This approach allows the system to handle unstructured input and generate flexible, context-aware suggestions.

In addition to the AI model, a rule-based matching system is implemented as a secondary component. This system uses ingredient normalization, fuzzy string matching, and a scoring logic to compare the user's input with recipes in the local dataset. Recipes are ranked based on ingredient overlap, and the highest-scoring matches are selected as suggestions. This approach is fast and consistent but limited to recipes that closely match the input.

The combination of these two approaches creates a hybrid system. The AI-assisted model improves flexibility and handles natural-language input, while the rule-based model provides reliable baseline results and a point of comparison. This design improves overall performance, robustness, and consistency of the system.

Evaluation

The system was evaluated using a variety of input cases to test its performance under different conditions.

When users provide multiple ingredients, the system performs well and generates relevant and realistic meal suggestions. For example, inputs such as “eggs and rice” consistently produce appropriate results like fried rice or omelettes.

However, performance decreases with limited input. When only a single ingredient is provided, the rule-based system may fail to return results due to strict matching thresholds. Additionally, ingredients not present in the local dataset are not considered by the rule-based system.

The AI model improves flexibility by handling a wider range of inputs and generating more creative suggestions, but it can occasionally produce unrealistic combinations. We evaluated the MVP using three representative cases: a single-ingredient input, a strong multi-ingredient input, and a messy natural-language input.

Test Input	No-AI Result	AI Result	Conclusion
eggs	weak/limited result	returned 3 suggestions	AI handled single ingredient better
eggs, rice, soy sauce, garlic, onion	returned 3 suggestions	returned 3 suggestions	both worked well
half an onion, leftover rice, some eggs, random sauces	partially handled messy input	returned 3 suggestions	AI handled casual language better

Overall, the results show that the AI-assisted model improves flexibility and handles unstructured input more effectively, while the rule-based system performs well when strong ingredient matches are available.

Limitations & Risks

The current MVP of PantryPal has several limitations that affect both system performance and user experience. One major limitation is the size of the local recipe dataset, which is relatively small and restricts the variety of possible matches. As a result, the rule-based system may struggle to find strong matches, especially when users provide uncommon or limited ingredients. Also, the system does not account for dietary restrictions, allergies, or nutritional considerations, which limits its usefulness for a wider range of users.

The rule-based approach also relies on strict ingredient matching, which can lead to weak or no results when the input does not closely align with the dataset. Ingredients that are not included in the dataset are not considered, further reducing the effectiveness of this approach in certain scenarios.

The AI-assisted model improves flexibility but introduces its own limitations. While it can handle unstructured input and generate creative suggestions, it can sometimes produce unrealistic or impractical meal ideas. Its outputs are also dependent on prompt quality and may vary across different inputs, leading to inconsistent results.

From a usability perspective, the system performs best when multiple ingredients are provided and may struggle with minimal input. There are also no advanced filtering options, personalization features, or user preference settings in the current version.

There are additional risks related to system dependency and scalability. The AI component relies on an external API, which introduces potential issues such as network delays, response variability, and cost considerations. These limitations highlight areas for future improvement in both the technical system and overall user experience.

Next Steps

With additional development time, several improvements could be made to enhance both the technical performance and usability of PantryPal.

On the technical side, the system could be expanded to include a larger and more diverse dataset from sources such as *Kaggle* or *Hugging Face*. This would improve the coverage and quality of the rule-based matching system. More advanced matching techniques, such as embedding-based semantic similarity, could also be implemented to better handle partial matches and improve accuracy when user input does not directly align with the dataset. In addition, the AI pipeline could be enhanced with multi-step processing and improved prompt design to increase consistency and reduce unrealistic outputs which would drastically improve the product.

From a product perspective, new features could include dietary filters, allergy considerations, and minimizing missing ingredients in generated recipes. Users could also be given the ability to save, rate, or customize meal suggestions. The interface could be further improved for mobile compatibility and overall usability to create a smoother user experience, such as getting a notification that something has expired and you need to restock.

Finally, a more formal evaluation process could be implemented using a larger and more structured set of test cases, along with real user feedback. This would allow for better measurement of system performance and help guide future improvements. With these enhancements, PantryPal could evolve into a more robust, accurate, and user-friendly application.

Conclusion

PantryPal demonstrates a functional MVP through two implementations: a rule-based baseline and an AI-assisted version. Testing both approaches showed that the AI-assisted model performs better when handling limited and unstructured inputs, while the rule-based system provides consistent and reliable results when strong ingredient matches are available.

While the system has several limitations, it successfully demonstrates the core idea of transforming simple ingredient input into useful meal suggestions. The hybrid design highlights the benefits of combining traditional methods with modern AI techniques.

Overall, PantryPal provides a strong foundation for future development and illustrates the potential of AI-assisted decision making in everyday tasks such as meal planning.

Demo Evidence and Test Cases

The following screenshots show the app interface and the main test cases used to compare the no-AI baseline and the AI-assisted version.

1. **Input screen**
   PantryPal homepage with the ingredient box visible.

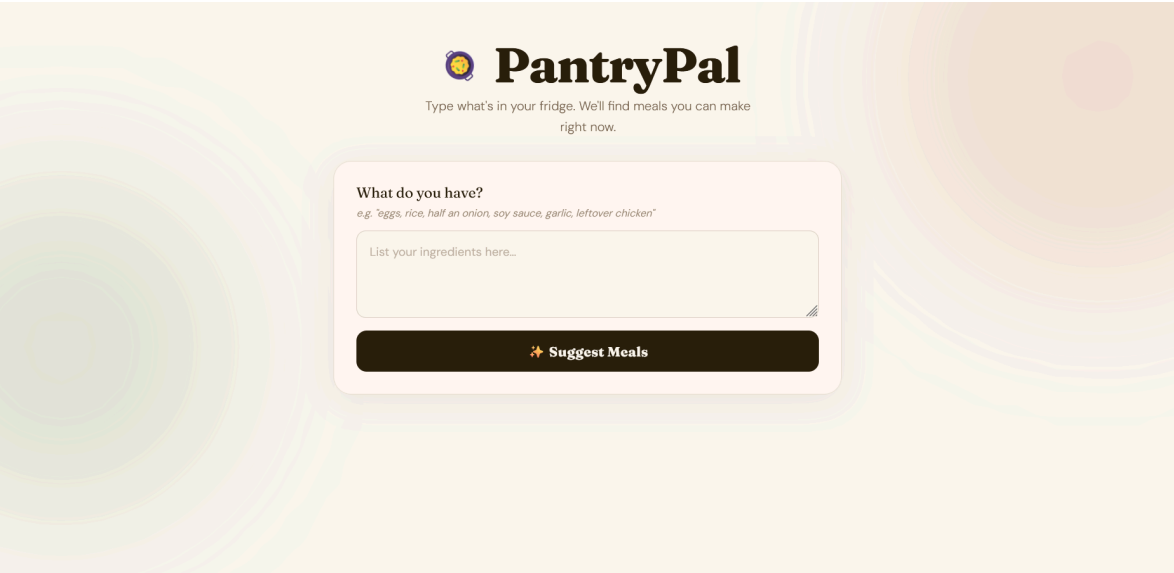

Figure 1. PantryPal input screen

2. **Successful result**
   Enter: eggs, rice, soy sauce, garlic, onion

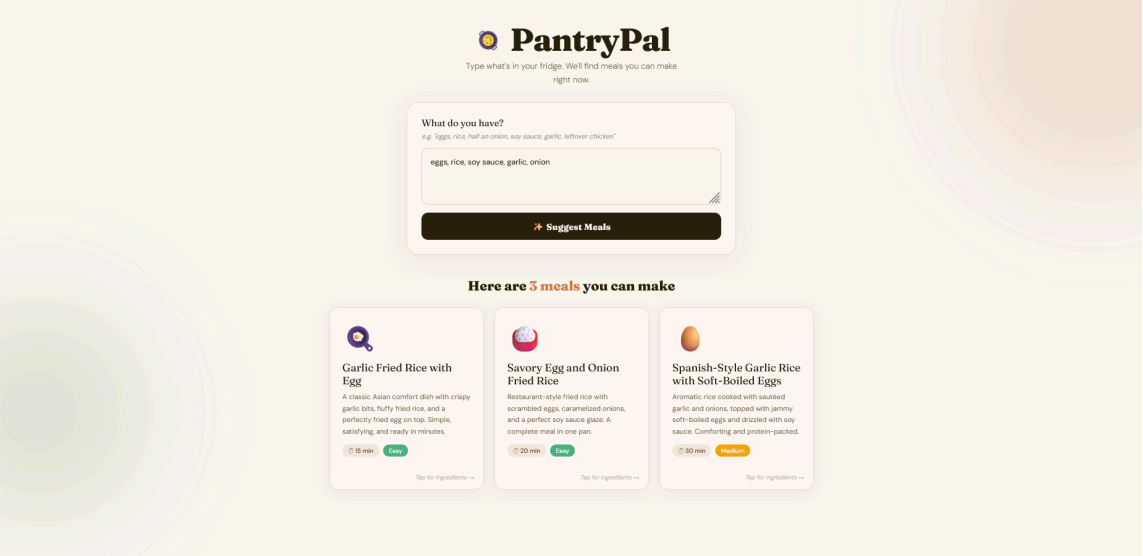

Figure 2. Successful result for “eggs, rice, soy sauce, garlic, onion”

3. **Weak/failure case**
   Enter: eggs
   Screenshot the result, even if it is bad.

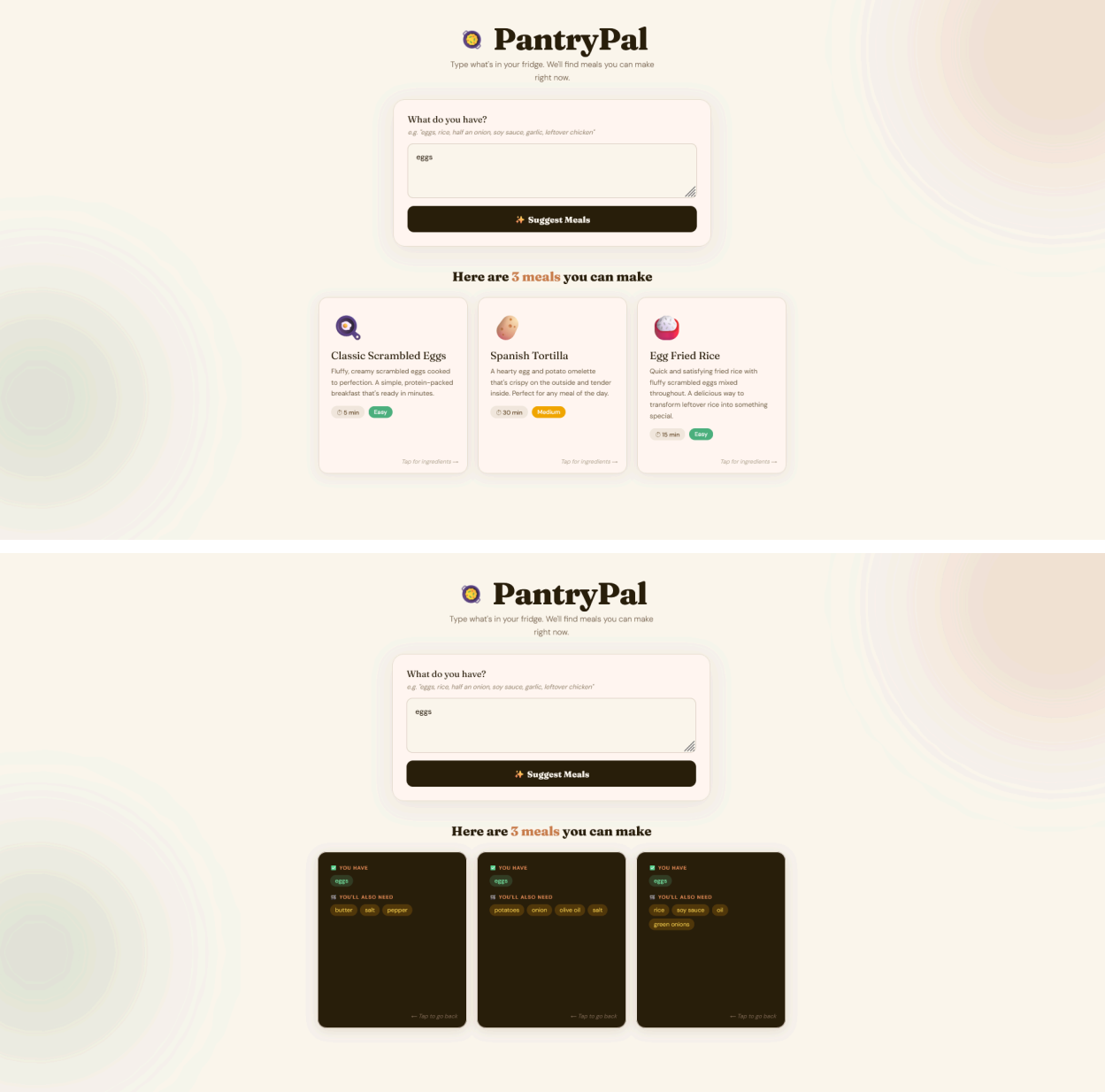

Figure 3. Single-ingredient test using “eggs”

4. **Messy input case**

   Enter: half an onion, leftover rice, some eggs, random sauces

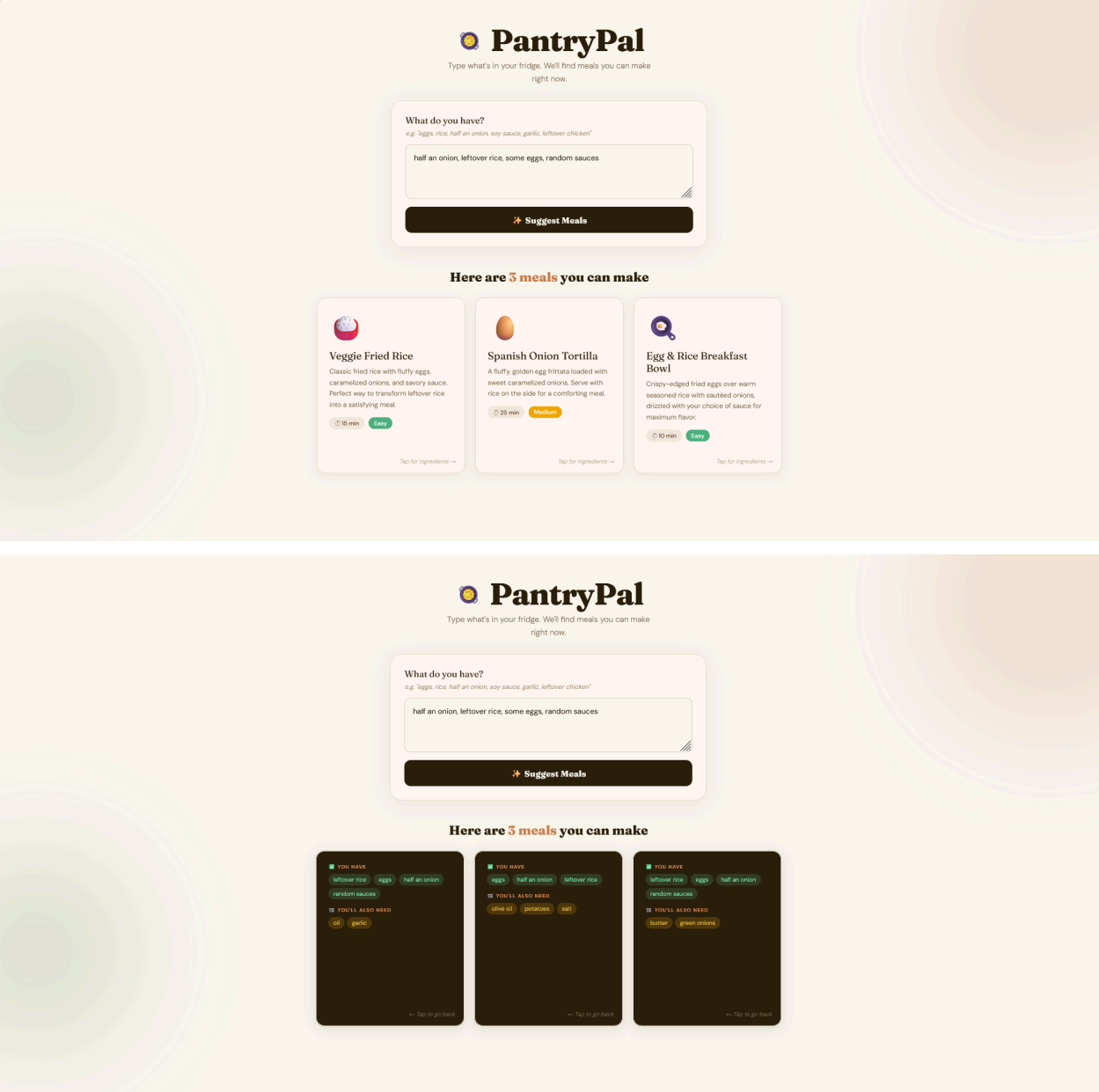

Figure 4. Messy input test using casual language

In our small test set, the AI-assisted version returned three suggestions for every input we tested. This suggests that it handled limited and messy inputs better than the no-AI version. However, more testing would be needed before claiming that the AI version is reliable for all possible ingredient inputs.

PantryPal MVP Demo Instructions

Overview

PantryPal is a web-based meal suggestion app. Users enter ingredients they already have, and the app returns three meal suggestions with descriptions, used ingredients, missing ingredients, cooking time, and difficulty.

Environment Setup

Install the required Python packages:

```
pip3 install flask anthropic
```

Running the No-AI Baseline Version

The no-AI baseline does not require an API key.

If the file is in the main project folder, run:

```
python3 pantrypal_noai.py
```

Then open a browser and go to:

```
http://localhost:5000
```

Running the AI-Assisted Version

The AI-assisted version requires an Anthropic API key.

Set the API key in the terminal:

```
export ANTHROPIC_API_KEY=your_api_key_here
```

If the file is in the main project folder, run:

```
python3 pantrypal.py
```

Then open a browser and go to:

```
http://localhost:5000
```

Minimal Demo

To test the app, enter:

```
eggs, rice, soy sauce, garlic, onion
```

The app should return three meal suggestions.

Notes

The no-AI version is easiest to run because it does not require an API key. The AI-assisted version can handle more flexible natural-language input, but it requires a valid Anthropic API key.